CS 39006: Computer Networks Laboratory

POSIX Network Programming: Concurrent Servers

Department of Computer Science and Engineering

INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

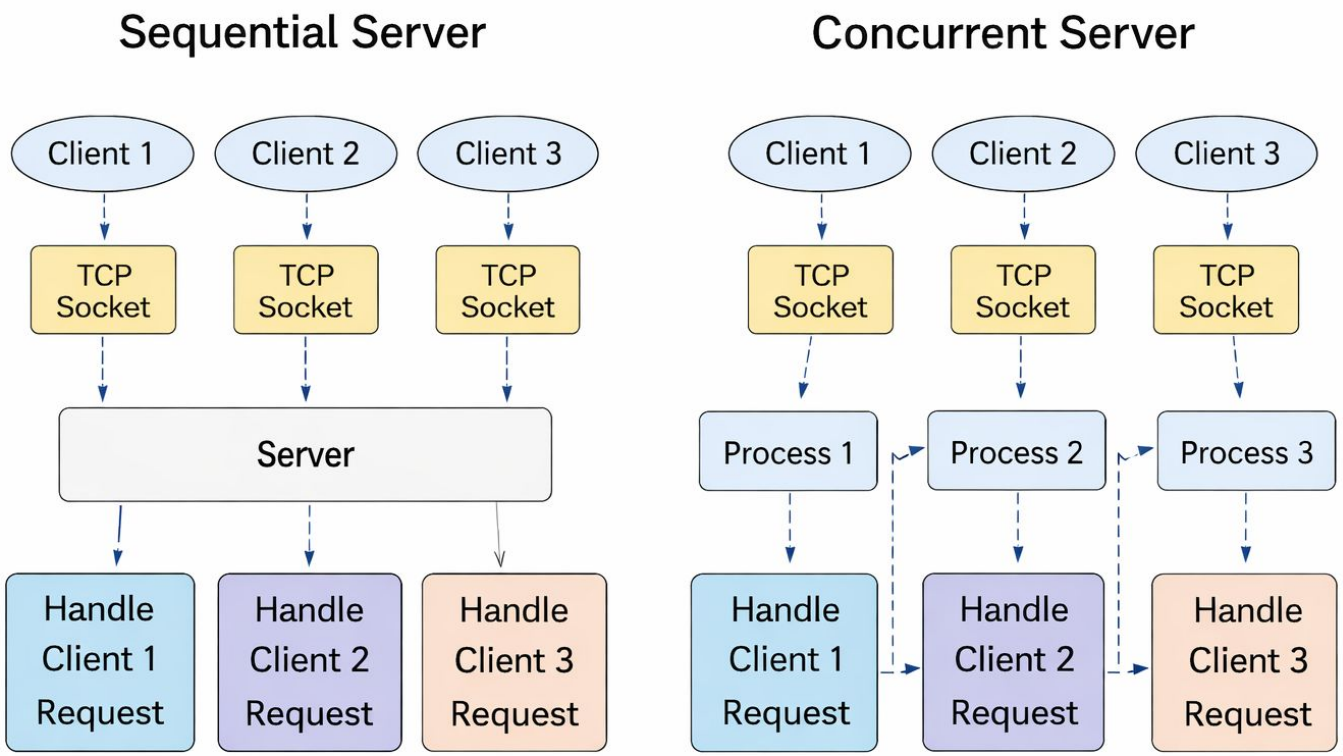
Abhijnan Chakraborty
abhijnan@cse.iitkgp.ac.in

Sandip Chakraborty
sandipc@cse.iitkgp.ac.in

Concurrent Servers (Vs Sequential Servers)

Key Characteristics

- A server capable of handling multiple client connections simultaneously.
- Clients should not wait for each other; each interaction proceeds independently.
- Needed for scalable network services (chat servers, web servers, etc.).



Problem: Blocking accept()

- `accept()` blocks until a new client connects.
- While blocked, the server cannot handle any other client.
- A single-threaded blocking server becomes unusable under multiple clients.

```
bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));
listen(sockfd, 5);

len = sizeof(client_addr);
client_sock = accept(sockfd, (struct sockaddr*) &client_addr, &len);
```

Problem: Blocking accept()

- `accept()` blocks until a new client connects.
- While blocked, the server cannot handle any other client.
- A single-threaded blocking server becomes unusable under multiple clients.

What we need to handle blocking issue?

- To avoid blocking on one operation and ignoring others.
- To allow multiple simultaneous connections.
- **Two mechanisms:**
  - process-based concurrency (`fork`)
  - I/O multiplexing (`select`).

Approach 1: `fork()` for Concurrency

- Each new client connection handled in a separate child process.
- Parent continues accepting new clients immediately.
- Each child blocks independently on its own socket.

How it works?

- Parent: `accept()` new client → `fork()` a child.
- Child: serves the client and exits when done.
- Parent: closes client socket and returns to `accept()`.

fork() for Concurrency

```
listen_fd = socket(AF_INET, SOCK_STREAM, 0);
//Populate server address for bind
...
bind(listen_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
listen(listen_fd, 5);

printf("Server listening on 8080...\n");

while (1) {
    client_fd = accept(listen_fd, (struct sockaddr *)&client_addr, &len);
    if (client_fd < 0) continue;

    pid_t pid = fork();
```

fork() for Concurrency

```
if (pid == 0) {
        // Child: handle this client
        close(listen_fd);

        char buffer[1024];
        int n = read(client_fd, buffer, sizeof(buffer));
        buffer[n] = '\0';
        printf("Child %d received: %s\n", getpid(), buffer);
        write(client_fd, "Hello from child process\n", 26);
        close(client_fd);
        exit(0);
    }
```

fork() for Concurrency

```
else {
        // Parent: go back to accept
        close(client_fd);
    }
}
```

fork(): Pros and Cons

- **Pros:** Simple model, blocking I/O is fine in children.
- **Pros:** Fault isolation – one client crash doesn't affect others.

- **Cons:** Expensive to create many processes.
- **Cons:** Not ideal for very high concurrency.

Approach 2: `select()` for Concurrency

- Single process monitors many sockets using `select()`.
- `select()` wakes up when any socket becomes readable.
- Server checks all ready sockets and serves them one by one.

How It Works?

- Maintain a set of active sockets including the listening socket.
- Call `select()` with this set.
- If listening socket ready → accept new client.
- If a client socket ready → read and respond.

select() Function Call

A multiplexing system call that allows a server to monitor **multiple file descriptors** (FDs) (sockets, stdin, pipes, etc.) simultaneously and wake up only when **at least one becomes ready for I/O**.

```
int select(int nfds,
           fd_set *readfds,
           fd_set *writefds,
           fd_set *exceptfds,
           struct timeval *timeout);
```

select() Function Call

A multiplexing system call that allows a server to monitor **multiple file descriptors** (FDs) (sockets, stdin, pipes, etc.) simultaneously and wake up only when **at least one becomes ready for I/O**.

```
int select(int nfds,
           fd_set *readfds,
           fd_set *writefds,
           fd_set *exceptfds,
           struct timeval *timeout);
```

nfds

The **largest file descriptor + 1** among all the FDs you want to monitor.
This tells the kernel how many descriptors it needs to check.

select() Function Call

A multiplexing system call that allows a server to monitor **multiple file descriptors** (FDs) (sockets, stdin, pipes, etc.) when **at least one becomes ready for I/O**.

```
int select(int nfds,
           fd_set *readfds,
           fd_set *writefds,
           fd_set *exceptfds,
           struct timeval *timeout);
```

readfds

Set of FDs monitored for **read readiness**.
A socket becomes readable when:

- new incoming data is available
- connection closed (read returns 0)
- listening socket receives a new connection (ready to `accept()`)

select() Function Call

A multiplexing system call that allows a server to monitor **multiple file descriptors** (FDs) (sockets, stdin, pipes, etc.) simultaneously and wake up only when **at least one becomes ready for I/O**.

```
int select(int nfds,
           fd_set *readfds,
           fd_set *writefds,
           fd_set *exceptfds,
           struct timeval *timeout);
```

writefds

- Set of FDs monitored for write readiness.
- Used mostly for non-blocking sockets or large buffer writes.

select() Function Call

A multiplexing system call that allows a server to monitor **multiple file descriptors** (FDs) (sockets, stdin, pipes, etc.) simultaneously and wake up only when **at least one becomes ready for I/O**.

```
int select(int nfds,
           fd_set *readfds,
           fd_set *writefds,
           fd_set *exceptfds,
           struct timeval *timeout);
```

exceptfds

Monitors “exceptional conditions” (rarely used for basic TCP lab work).

select() Function Call

A multiplexing system call that allows a server to monitor **multiple file descriptors** (FDs) (sockets, stdin, pipes, e
when **at least one becomes ready for I/**

```
int select(int nfds,
           fd_set *readfds,
           fd_set *writefds,
           fd_set *exceptfds,
           struct timeval *timeout);
```

timeout

Controls how long `select()` waits:

- NULL → wait forever
- (0,0) → return immediately (polling)
- Custom timeval → wait up to t seconds

How `select()` Works

1. You fill FD sets using `FD_SET(fd, &set)`.
2. You call `select()`.
3. Kernel pauses your process until any FD becomes ready.
4. Kernel wakes you up and modifies the FD sets to keep only ready descriptors.
5. You check each one using `FD_ISSET(fd, &set)`.

Example Server Socket

```
int listen_fd, client_fd;
struct sockaddr_in server_addr, client_addr;
socklen_t len = sizeof(client_addr);

fd_set master_set, read_set;   // will be used in select()
int max_fd;

listen_fd = socket(AF_INET, SOCK_STREAM, 0);
if (listen_fd < 0) {
    perror("socket");
    exit(1);
}
```

Example Server Socket

```
memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family      = AF_INET;
server_addr.sin_port        = htons(8080);
server_addr.sin_addr.s_addr = INADDR_ANY;
if (bind(listen_fd,(struct sockaddr*)&server_addr, sizeof(server_addr))< 0) {
    perror("bind");  exit(1);
}

if (listen(listen_fd, 5) < 0) {
    perror("listen"); exit(1);
}

printf("Server listening on 8080...\n");
```

Example Server Socket

```
FD_ZERO(&master_set);
FD_SET(listen_fd, &master_set);
max_fd = listen_fd;

while (1) {
    read_set = master_set;        // select() modifies the set, so use a copy

    int ready = select(max_fd + 1, &read_set, NULL, NULL, NULL);
    if (ready < 0) {
        perror("select");
        break;
    }
```

Example Server Socket

```
// 1. Check if there is a new incoming connection
   if (FD_ISSET(listen_fd, &read_set)) {
       client_fd = accept(listen_fd, (struct sockaddr *)&client_addr, &len);
       if (client_fd < 0) {
           perror("accept");
       } else {
           printf("New client connected: fd=%d\n", client_fd);
           FD_SET(client_fd, &master_set);
           if (client_fd > max_fd) max_fd = client_fd;
       }
   }
```

Example Server Socket

```
// 2. Check all existing client sockets for data
  for (int fd = 0; fd <= max_fd; fd++) {
      if (fd == listen_fd) continue;       // already handled

      if (FD_ISSET(fd, &read_set)) {
          char buffer[1024];
          int n = read(fd, buffer, sizeof(buffer) - 1);
          if (n <= 0) {
              // client closed or error
              if (n < 0) perror("read");
              printf("Client on fd %d disconnected\n", fd);
              close(fd);
              FD_CLR(fd, &master_set);
```

Example Server Socket

```
            } else {
                    buffer[n] = '\0';
                    printf("Server received from fd %d: %s\n", fd, buffer);
                    const char *msg = "Hello from select() server\n";
                    write(fd, msg, strlen(msg));
                }
            }
        }
    }

    close(listen_fd);
    return 0;
}
```

`select()`: Pros and Cons

- **Pros:** No process creation overhead.
- **Pros:** Efficient for moderate concurrency.

- **Cons:** More complex programming.
- **Cons:** Limited by `FD_SETSIZE`.